DE LA SALLE UNIVERSITY-DASMARIÑAS
COLLEGE OF ENGINEERING, ARCHITECTURE, AND TECHNOLOGY
ENGINEERING DEPARTMENT
COMPUTER ENGINEERING PROGRAM

SOFTWARE DESIGN

(T-CPET3228LA**)**

Quiz 1 – Chapter I-III

Grade

Submitted By:

NAAG, Tyrone Z.

MONROY, Adrian M.

Submitted To:

Engr. Michael C. Olivo
Professor

Architectural Design and Development of LocalHustle: A Microservices-Oriented Local Marketplace Supporting Community-Based Sellers

CHAPTER 1

INTRODUCTION

This chapter presents the introduction of the study. It includes the background of the study, the statement of the problem, the objectives of the study, the significance of the study, the scope and limitations, and the definition of key terms. These sections provide an overview of the research and its purpose.

1.1 Background of the Study

Electronic commerce has significantly impacted the way we buy and sell things today. Where brick and mortar transactions were previously completed, people can now exchange money for goods or services through online stores or websites. Global e-commerce sales have been increasing steadily as more people turn to purchasing items online (International Trade Administration, 2024). E-commerce platforms allow consumers access to many products and services offered through personal electronic devices which change consumer behavior (Sandu, 2025). Although reaching a broad audience is effective and convenient, e-commerce platforms tend to focus on mass audiences as opposed to offering a structured system for local buys and sells.

In contrast, unlike in the case of big e-commerce businesses where integrated logistics play an important role, in the case of digital marketplaces, the role of the platform is often that of an intermediary in peer-to-peer transactions. In such cases, the platform is used as a medium to allow the buyers and sellers to communicate and the transaction happens through direct contact between the two parties. This is essentially because this model is based on the way micro-

entrepreneurs and people in communities function and operate, rather than through large-scale transaction and fulfillment models. Developing a digital marketplace that facilitates this model of commerce requires significant emphasis to be given to the geographical relevance.

As digitalization continues to rise, small businesses and individual sellers look for platforms that enable them to compete without being overshadowed by high-volume commercial vendors. However, if localness is used as an auxiliary filter, small sellers may become invisible in highly saturated list environments. A system that enables geographic segmentation as an integral part can create a more balanced marketplace environment where proximity can be used as an organizing principle rather than an auxiliary filter tool.

Most casual vendors attempt to sell items using whatever methods they can find. A lot of people will use Facebook Marketplace or other implicit social media selling features. However, by allowing both business and non-business postings, these websites dilute the efficacy of the products being posted. Categorized postings, verification systems, and a clean interface dedicated to buying and selling are all fundamental to a high-quality marketplace (Laudon & Traver, 2020). Improper structuring can lead to a lack of standardized listing creation, no verification of sellers, and poor backlog management from administrators.

As online commerce is constantly advancing, issues related to trust and system governance have emerged as a focal point. For example, customers are looking for transparency in product details, clarity in seller details, and uniformity in product listing styles. Systems without a standardized created process or a structured administration process might suffer from varying degrees of data quality, which might influence system stability as well as user trust. In local commerce, these factors assume a greater degree of importance due to implicit trust within a community. A structured administration process enhances system integrity as well as user trust.

In terms of software engineering, when an application is built using only one code base, it can become unmanageable as the application continues to scale. Many new-age distributed systems are turning to a microservice architectural style, where an application is split up into several services that each handle their own unique business capabilities. Microservices allow for a better modular design, encapsulation of failures, and independent service deployment of specific capabilities needed to power a web-scale application. There have been many real-world cases from companies like Amazon and Netflix that have service separation. Building a marketplace platform does not only involve user interface development but also a complete architectural design, along with database separation and communication services. The application of a microservices-based approach allows for the separation of managements into different components, allowing them to be independent from each other.

In an e-commerce system, architectural modularity directly supports operational specialization. User authentication services, product listing services, search and filtering services, and order processing services are specialized with unique responsibilities and data management requirements. Specializing these components allows the system to accommodate increasing user activity without impacting other components that may be unrelated. For instance, location filtering mechanisms may be specialized independently of seller verification mechanisms, ensuring that geographic isolation does not impact transaction processing or data consistency. Such specialization is especially important for e-commerce systems that emphasize structural locality, where geographic data management and filtering must be efficient even when the system is scaled.

Through the integration of structured e-commerce principles and a service-oriented architecture, the platform will be able to maintain both organizational organization and system scalability, thereby providing the localized marketplace platform with the ability to present the products in a defined geographical

boundary, thereby allowing proximity to act as the main organizational principle, thereby providing the platform with the ability to address the technological evolution of e-commerce systems and the economic behavior of trade in the community.

1.2 Objectives of the Study

1.2.1 General Objectives

To design and develop a microservices-based local marketplace web application that enables community sellers to display and manage their products.

1.2.2 Specific Objectives

Objectives the researchers would like to achieve:

1. To design a modular architecture separating user management, product listings, and order processing services.
2. To develop a product module that allows verified sellers to create and manage listings.
3. To design a buyer module for browsing products and communicating with sellers.
4. To develop an admin module for seller verification, category management, and activity monitoring.
5. To design a secure authentication and verification system that restricts access to authorized users and administrators through validated login and role-based access control.

1.3 Significance of the Study

Throughout the years, with the rapid advancement of technology and communication services, comes the surge of various digital platforms. These platforms, ranging from social media networks to e-commerce websites, have revolutionized and shaped the way individuals connect, businesses operate, and data is exchanged (Bokoliya, 2023). Such developments further expedite the rise and continuous evolution of digital marketplace platforms, particularly in the fields of architectural design and deployment strategies. In response to such trends, this study introduces LocalHustle, a microservices-oriented local marketplace that combines modern architectural principles with community-centered commerce. Through the combined effect of localized economic engagement and scalable system design, the research examines the technical and socio-economic facets of platform development.

The expected primary beneficiary of this study and its proposed system are the community-based sellers, as its core functionalities are tailored to deal with the challenges that sellers encounter in general-purpose online marketplaces such as Facebook marketplace. These platforms often display a lack of standardized listing formats and verification systems, which often ends in inconsistent product information and limited seller credibility. For many small-scale entrepreneurs, the digital marketplace can feel like a landscape of uncertainty. Local vendors often struggle against a tide of skepticism, where a few bad actors can unfairly damage the reputation of honest sellers (Haganus et al., 2017). This study introduces features like seller verification and administrative oversight not just as technical hurdles, but as digital 'trust-builders.' By creating a safer, more organized space, the system helps local business owners stand out and connect with customers who feel secure in every transaction.

Beyond the advantages this platform offers sellers, local consumers also benefit greatly from it since it reduces the daunting digital marketplace into a comfortable and familiar community hub that places an emphasis on transparency and convenience. Consumer trust is increased and vendor credibility is reinforced through the incorporation of verified seller profiles and direct communication features. By encouraging localized transactions, the platform promotes a more sustainable kind of community-based commerce in which purchases support small businesses rather than just big, centralized firms.

From a software engineering standpoint, the use of a microservices-based architecture highlights the practical benefits of modular system decomposition in commercial applications. In comparison to traditional monolithic architectures, the system enhances scalability, maintainability, and fault isolation by breaking out user management, product listing, order processing, and administrative activities into distinct services. This architectural solution not only addresses performance and adaptability concerns, but it also serves as a documented case study for turning distributed systems theory into a practical web-based platform. The study provides practical insights into the design of scalable digital marketplace systems by articulating service boundaries, inter-service communication mechanisms, and deployment issues.

Along with its architectural and social contributions, the study has academic and institutional relevance. The formation of the system exhibits how distributed systems principles, modular service design, and marketplace platform engineering may be applied in a real-world context. For pupils as well as professionals in the field of computer engineering and related disciplines, the project demonstrates how theoretical ideas in software design may be translated into functional, scalable systems. Furthermore, the documented design decisions and implementation approaches can serve as the foundation for future research into cloud-native

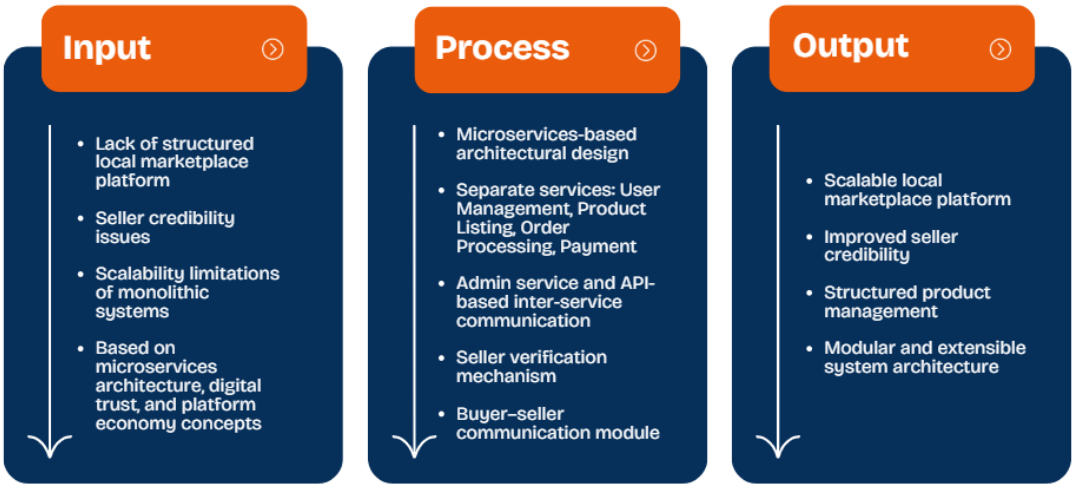
systems, platform governance, and scalable web technologies.

1.4 Scope and Limitation

This study focuses on the architectural design and development of LocalHustle, a microservices-based local marketplace platform designed to support community-based sellers. The system is conceptually built to scale across various regions of the Philippines using modular service decomposition and separate deployment strategies. Core features include user administration, seller verification, product listing management, buyer-seller communication, order processing, and administrative control. The study emphasizes the application of microservices principles such as service separation, inter-service communication, and modular deployment. Functional testing and performance evaluation are carried out in a controlled development environment to determine system behavior and architectural viability.

Despite the fact that the system architecture is intended to provide nationwide scalability, deployment, participant recruitment, and system testing are restricted to certain localities within the CALABARZON region due to time and resource constraints. While the platform incorporates a third-party payment gateway, financial transactions are carried out under controlled testing conditions and do not represent large-scale commercial deployments. This study does not address enterprise-level infrastructure scaling, high-volume traffic optimization, or nationwide user uptake. The system's academic implementation acknowledges regulatory, taxes, and large-scale financial compliance concerns, although they are not thoroughly handled.

1.5 Conceptual Framework



The conceptual framework for this study is based on digital marketplace theory, microservices architecture principles, and trust-based e-commerce models. The framework begins by addressing present challenges in general-purpose online marketplaces, such as seller reliability, a lack of organized categorizing, and the scalability constraints of monolithic systems. The paper proposes building a distributed marketplace platform using microservices design features such as modularity, service deconstruction, and independent deployment. The system intends to address identified marketplace inefficiencies by implementing discrete services for user administration, product listing, order processing, payment handling, and administrative monitoring. The desired objective is a scalable and structured local marketplace that builds confidence, supports community-based merchants, and encourages localized digital commerce.

1.6 Definition of Terms

Administrative Module
- A system component responsible for managing seller verification, category configuration, monitoring activities, and maintaining overall marketplace governance.

Application Programming Interface (API)
- A set of rules and protocols that enables communication between different software services within the microservices architecture and between the system and external platforms.

Buyer Module
- A system component that allows registered users to browse product listings, communicate with sellers, and initiate purchase transactions.

Community-Based Sellers
- Small-scale entrepreneurs or individual vendors operating within a defined geographic locality who utilize the platform to display and manage products.

Digital Marketplace
- An online platform that facilitates buying and selling activities by serving as an intermediary between buyers and sellers.

Inter-Service Communication
-The exchange of data and requests between independent services within the microservices-based system through APIs.

Microservices Architecture
- A software architectural approach in which an application is developed as a collection of independent, loosely coupled services, each responsible for a specific business capability.

Modular Architecture
- A system design approach that separates functionalities into distinct components to enhance scalability, maintainability, and independent deployment.

Payment Gateway

- A third-party financial service integrated into the system to process and validate electronic transactions between buyers and sellers.

Scalability

- The capability of the system to handle increasing workloads, users, or transaction volumes without compromising performance.

Service Decomposition

- The process of dividing a system into smaller, independent services based on business functions.

CHAPTER 2

REVIEW OF RELATED LITERATURE

This chapter presents the review of related literature and studies that serve as the foundation for the development of the proposed system. The study focuses on the application of the e-commerce system, which is intended for the buying and selling activities of the members of the community, as well as small business enterprises and common vendors.

2.1 Electronic Commerce Systems

E-commerce can be defined as the use of electronic networks, especially the internet, for carrying out commercial transactions involving the exchange of goods and services. Laudon & Traver (2020) note that modern e-commerce sites use different technology components such as digital storefronts, transaction management systems, payment mechanisms, and communication tools for facilitating commercial transactions. The use of these sites allows buyers to access different goods and services and perform commercial transactions without being confined by geographical boundaries.

Additionally, these sites offer opportunities for small-scale commercial activities and individuals to participate in commercial activities that were previously dominated by large organizations. The flexibility and scalability of these sites allow commercial organizations to expand their operational boundaries through the use of mobile and web-based applications. Laudon & Traver (2020) note that the success of e-commerce sites depends on their capability to match the characteristics and requirements of the target users. This implies that commercial organizations need to design specific sites that meet the requirements of local markets rather than relying on general large-scale commercial activities.

The rapid growth in the volume of international e-commerce transactions is a clear manifestation of the increasing trend towards the use of digital technology in facilitating commercial transactions. According to the International Trade Administration (n.d.), the volume of international e-commerce transactions is still growing due to improvements in digital technology, increased access to the internet, and the popularity of mobile technology. The trend towards increased consumer engagement in online commercial activities is a manifestation of the advantages associated with these activities.

The growth in international e-commerce transactions suggests that the use of centralized commercial platforms may overlook localized commercial activities that focus on proximity and community engagement. This suggests the potential benefits associated with the development of marketplace technology.

2.2 Digital Platforms and Local Economic Development

Digital platforms have emerged as key players in influencing local economic activities by providing support for new styles of entrepreneurship and business

participation. Bokoliya (2023) asserts that digital platforms can play a role in local economic development by allowing individuals to market their products or services within their local geographical settings. This increases income opportunities, improves market accessibility, and increases participation among vendors who may not have the means to develop retail businesses.

Furthermore, digital platforms for local activities enhance peer-to-peer economic interactions that are not based on centralized commercial institutions. By allowing individuals to engage in economic activities with each other, digital platforms strengthen local-level economic networks and encourage cooperative participation. However, challenges are also identified by Bokoliya (2023) as possible barriers to the role of digital platforms in fully supporting local economic development.

2.3 Location-Based Digital Commerce

The addition of location-based services in digital platforms has further increased the importance of localized marketplace systems. According to Zhang, Li, & Wang (2020), proximity-aware technology helps users identify products or services located in proximity to them. This improves the convenience of transactions by reducing the complexity involved in logistics. By incorporating location-based services into digital commerce platforms, users are provided with personalized search results. This helps them efficiently interact with each other in a particular geographic area.

The proximity-based system has also enabled the development of flexible transaction systems like meet-ups or delivery services over a short distance. This has reduced transaction costs while improving user satisfaction by allowing immediate transactional interactions. Location-based commerce is a significant technology for the development of a community-based marketplace platform.

2.4 Peer-to-Peer Marketplace Platforms

Peer-to-peer marketplaces have been recognized as important facilitators of decentralized economic participation. In this regard, Hamari, Sjöklint, and Ukkonen (2020) explained that sharing economy platforms enable individuals to engage in direct exchange of goods and services, thus facilitating collaborative consumption. These platforms enable individuals to assume both roles of a buyer and a seller, thus facilitating inclusive participation and social interaction in digital commerce environments. The community-driven marketplaces offer opportunities for informal entrepreneurship, where individuals can benefit from their underutilized resources or skills. The success of these marketplaces depends on their accessibility, usability, and ability to create socially embedded transaction experiences that match localized patterns of trading.

2.5 Microservices Architecture in Web Applications

As new features and services are added over time, modern software systems continue to expand in size and complexity. A monolithic design, in which the entire application is written and deployed as a single unit, was commonly employed in the development of traditional systems. Shabani (2022) states that this architectural approach usually consists of a single codebase with closely linked components including database management, business logic, and user interface. While monolithic systems are relatively simple to develop in the early stages of a project, they can be difficult to maintain and scale as the application grows as each component highly depends on one another and is integrated into a single framework.

Many contemporary software systems use microservices architecture to overcome these constraints. In this type of structure, the main program is divided

into smaller, independent services instead of coding in a single codebase. In addition, each of these services carry out a specific function of the system and communicate with one another through the use of techniques such as application programming interfaces (APIs). Terdal (2022) states that a microservices architecture enables developers to continuously integrate and deploy changes to the application without causing any downtimes thus, making it also resilient to failure.

To elaborate further, this architecture tends to improve the reliability and the scalability of the system simply due to the fact that individual services can be scaled depending on their specific workload requirements. A solid example of these are services such as user authentication or product listings would most likely require different computing resources when compared to other components. Such independent scaling capacity basically enables system resources to be handled more efficiently and often translates to a better maintenance of the system's performance even as the number of users increases (Shabani, 2022; Terdal, 2022).

Furthermore, research has also highlighted how essential monolithic system decomposition truly is. Abgaz et al. (2023) provided a more intricate description on the process of system decomposition wherein large applications are divided into multiple independent services, each executing their specific functional responsibility. Such an approach makes complex systems easier to maintain and track given that it allows components to be developed or deployed independently, as well as increase its resiliency towards failure as this means that one system failing would not necessarily affect the operation of the other services deployed in the application.

In respect to the advantages mentioned, the utilization of microservices architecture has been preferred in creating modern web platforms and distributed

applications. This architectural approach which features modular services fits perfectly for complicated platforms such as e-commerce and marketplace systems that require different services such as user management, product listing and order processing, as well as the fact that they are designed to accommodate large numbers of users.

CHAPTER 3

METHODOLOGY

In this chapter, the methodology used in the development of the LocalHustle website will be discussed. This includes the outlines of the overall system design, development approach, and technologies utilized in building the platform. It discusses the processes involved in user authentication, seller application and admin review, listing management, and user interaction features such as browsing, saving, and reporting listings. Additionally, this chapter explains the database structure, system architecture, and testing procedures conducted to ensure the functionality, reliability, and security of the system.

3.1 Methodological Framework

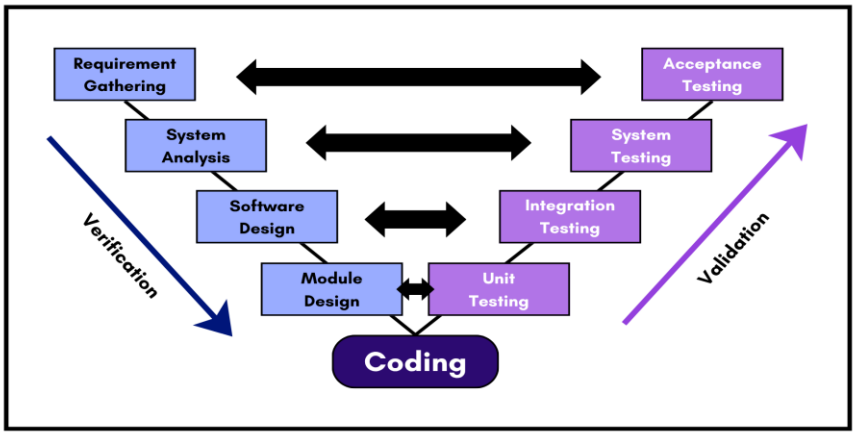


Figure 3.1.1 V-Shaped Software Design Life Cycle Model

For this proposed system, it employs the V-Shaped Software Development Life Cycle (SDLC) model given that it is done in phases wherein each development stage has an equivalent testing period, a trait that the researchers find of importance especially when it comes to user data handing, messaging and as well as interactions relating to marketplace which typically involves money. To begin, in the requirements analysis phase, key features were identified namely: login, signup, seller applications, management and display of listings, reporting, notifications, chatting system, meetup mechanics as well as other planned features for transactions and social media feed. Such requirements give a clear outline of what the system should be capable of and would also function as a basis for later testing to check if each function is working correctly. Moving on, in the system design phase, this is the part wherein we structurally divided the storage system of the system by turning it into modular components via separate databases, giving one for each service. Although not implemented fully, for the chat system and upcoming transaction and social feed features, REST-like PHP endpoints under the /api/ directory are used to emphasize application of microservices architecture. Apart from that, in the implementation phase of this project, features were sequentially done from applying authentication, seller application handing, displaying and managing listing up to future features involving transactions and socializing. PHP with sqlsrv and SQL server were utilized in this phase to guarantee compatibility and organization of database interaction.

In regard to the testing parts of each stage, each individual PHP files and endpoints were unit tested such as start_conversation.php, send_message.php and more to see if input validation, session handling, database interactions and JSON responses are working as intended. The next part would be integration testing, a stage that proved difficult as this is the part wherein different components interaction should be checked. For example, the flow from the

"Message Seller" button from each page containing it and how it should transition to conversation creation, message sending and retrieval and of course, the meetup proposal process. Now for system testing, the overall platform of LocalHustle was tested in the view of users, buyers and sellers alike, by checking listings, messaging, reporting contents, saving listings, receiving notifications and handling meetup proposals, all the while observing proper navigation, responsiveness and accessibility. Moving on, the acceptance testing helps confirm that each feature of the system meets the user's expectations while also validating seller workflows and admin actions. Lastly, inside the maintenance phase, future enhancements such as the usage of external APIs to implement real-time chat using Pusher, Google Maps integration for meetup locations and the addition of transaction and social feed modules are supported. Overall, the V-Shape SDLC model is appropriate for LocalHustle System as it emphasizes continuous validation and verification at each stage, a trait that is very important when handling marketplace and social media platforms.

3.2 Data Gathering Procedure

In this study, data will be gathered using a structured survey questionnaire based on a Likert scale, going from 1 – Strongly Disagree to 5 – Strongly Agree. This survey would be distributed to selected respondents that used the LocalHustle web application. Respondents include a number of experts as well as students or potential users accustomed to online marketplaces and social platforms to ensure the integrity and relevance of data gathered. In regard to this survey, a brief orientation will be first given to the respondents wherein the researchers will teach them how to navigate the said system, and afterwards, they will be asked to perform specific tasks such as creating an account, applying as a seller, sending messages, and more. Once the tasks are completed, the participants will answer

a survey designed to capture and reflect their actual experiences while using the system. With this method, the responses from participants would most likely be based on actual experience rather than blind assumptions. Another advantage of using Likert scale is that it enables the researchers to evaluate the users' feedback in a consistent manner, thus making it easier to analyze trends and check on areas that need improvement.

For the contents of the survey, it will focus on observable system parameters so that the data gathered will be clear and actionable. The system parameters it would tackle there are the website's responsiveness, layout consistency across pages, ease of moving through features, usage of forms, and accessibility. In addition, it would also get data regarding the reliability of message delivery, clarity of notifications, difficulty of setting up meet-up proposals, and, of course, data accuracy, especially when it comes to the data displayed on the dashboards. Lastly, the survey would also get feedback regarding session stability and the interaction between features. With this set of parameters, the researchers will be able to assess the system in detail rather than vague and general impressions.

3.3 Research Design

Developmental-descriptive research design is used in the process of designing, developing, and evaluating the LocalHustle application. In developmental research design, attention is paid to the process of creating new software products. Thus, the process of developing the LocalHustle web application and social media app included the following components: user authentication, processing of applications by sellers, listing, messaging, reporting, and transaction handling. By using this methodology, it became possible to develop

a web-based marketplace and social media application that could meet certain user needs and system requirements.

Descriptive design was applied in the evaluation of the usability, efficiency, and performance of the developed software product. After interaction with the LocalHustle platform, selected participants were invited to assess their experience via a survey questionnaire. Thus, it was possible to collect reliable data about user perceptions regarding the performance efficiency, usability, and functionality of the software product developed by the researchers. The proposed methodology can be characterized as an effective way to evaluate and develop new software products. It was used by the researchers to identify strong and weak points of the LocalHustle platform.

3.4 Research Instruments

The main data collection tool that was used in this study is the questionnaire which was developed in order to assess the usability, functionality, and effectiveness of the LocalHustle platform. The respondents, including both users (buyers and sellers) and administrators, completed the questionnaire after carrying out some tasks related to the use of the system. This questionnaire contained a series of statements associated with the criteria that were going to be used to make evaluations (that is, usability, functionality, performance efficiency, and user satisfaction). For each statement, the respondent had to choose an answer from the following five options (the number refers to the level of agreement): (1) strongly disagree; (2) disagree; (3) neutral; (4) agree; (5) strongly agree.

The usability aspect focused on how easy it was to navigate through the interface and whether the user interface was comprehensible. Functionality entailed the assessment of how well the system performed in terms of the functionalities provided such as the managing of listings, processing of applications

from sellers, messaging, reporting, and transactions handling. On the other hand, performance efficiency involved the speed of system operation and performance, whereas user satisfaction entailed how satisfied users felt about using the website. order to ensure that the questions were valid and relevant, the questionnaire items were constructed based on the key features of the LocalHustle system.

3.5 System Process and Design

3.5.1 System Architecture

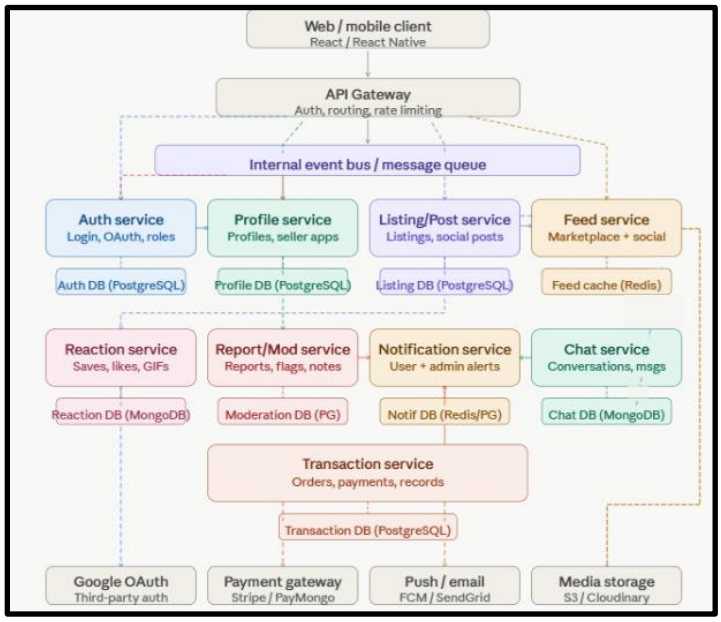


Figure 3.5.1.1 LocalHustle System Architecture

This diagram depicts the general architecture of the system architecture of LocalHustle platform that relies on microservices-based approach that offers scalability, flexibility, and efficient interaction among various components of the application. The top tier consists of clients that connect to LocalHustle via browser or mobile applications based on such frameworks as React or React Native. All requests from the client-side applications go through API Gateway, which acts as a single point of connection between the client and backend. The API Gateway

provides security services by managing requests and performing authentication and rate limiting.

As part of the microservice architecture, the event bus and message queue is located behind the API Gateway. This component enables effective interaction of the microservices within the system without direct interaction. Microservices are responsible for various application layers and perform specific actions according to their purpose. Such microservices can be considered as the core of the LocalHustle platform. Auth Service deals with the process of user authentication and authorization, and it operates in connection with PostgreSQL databases. Profile Service works with profiles and seller applications while Listing/Post Service is responsible for posting in marketplaces and in the social section.

Further additional services include the ones that provide functionality for other components within the platform. Specifically, the Reaction Service is responsible for liking and saving content on the site. Also, the Report/Moderation Service is in charge of moderating and managing reported content on the website. The Notification Service delivers notifications and integrates with external messaging platforms. Meanwhile, the Chat Service provides live chat opportunities to users.

The Transaction Service includes all the functionality related to orders, payments, and transactions and uses PostgreSQL database for storing information. Finally, the services that integrate with other third-party services, such as Google OAuth (used for third-party authentication), payment services (Stripe and PayMongo), notification services (push, email), and media storage services (Amazon S3, Cloudinary) are implemented.

In general, each individual service uses its own database and adopts a decentralized approach to data storage. In addition, integration services provide a way to incorporate external services in the platform architecture. With a proper

set of services, the LocalHustle application will be able to perform efficiently and provide reliable support for simultaneous user interactions.

3.5.2 Database Structure and Design

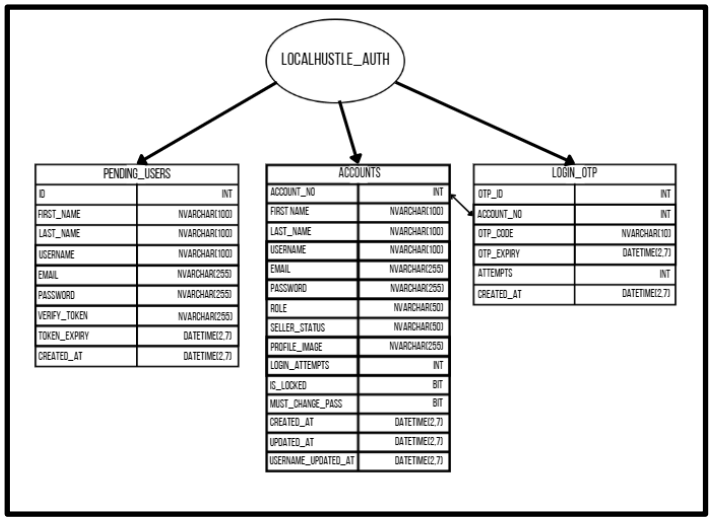


Figure 3.5.2.1 LocalHustle_Auth Database Structure

The figure is the database schema representation for the authentication component in the LocalHustle platform referred to as LOCALHUSTLE_AUTH. It contains mainly three different database tables namely; PENDING_USERS, ACCOUNTS, and LOGIN_OTP. Each of the tables has its unique functions for supporting the management of authentication and account security within the application. PENDING_USERS is the table that holds the details of all pending users within the platform who have registered with LocalHustle but have yet to go through the account verification process. This table holds basic user details such as name, username, email, and password, verify_token, and token expiry date. All these features make sure that only verified users can continue creating their accounts.

After account verification, users' data is transferred to another important table called ACCOUNTS, which holds details of all authentic users. In addition, it

also provides various security options including user credential data such as password, role, seller account, and profile picture. Other columns include data regarding login attempts, account lock status, and password reset requirement. Finally, LOGIN_OTP is another security table used for storing one-time password and its expiration and attempt number per account.

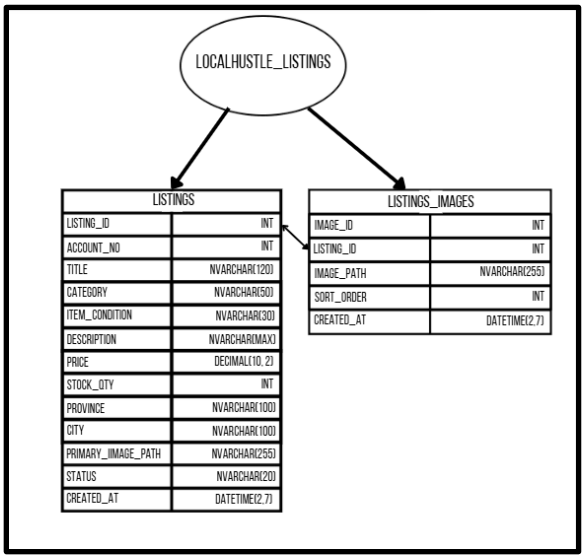


Figure 3.5.2.2 LocalHustle_Listings Database Structure

The figure above is the database model of the listings module of the LocalHustle system with the schema name LOCALHUSTLE_LISTINGS. The listing database schema consists of two major tables; these include the LISTINGS and the LISTINGS_IMAGES tables that will be used to store information about listings as well as the listing's related images.

The LISTINGS table is the primary table that holds all the necessary data of each listing. This table has the listing ID, account number, title, category, item condition, and description. More attributes like price, quantity, and location are included to complete the listing information. Also included are other fields like a primary image path to allow fast loading, listing status indicating whether the listing is active or not, and timestamp when the listing is created. In the

LISTINGS_IMAGES table, multiple images related to a listing are held using fields like image ID, listing ID, path to the image, and sort order for images display ordering. Other information like a timestamp indicating the time an image is uploaded is also provided.

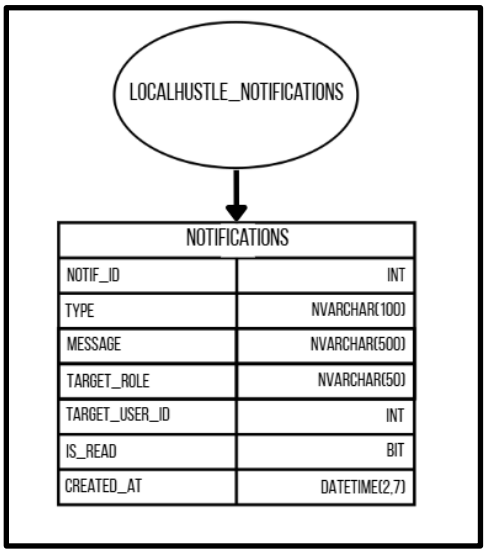


Figure 3.5.2.3 LocalHustle_Notifications Database Structure

The figure above is depicting the schema of the notification database within the LocalHustle application under the name LOCALHUSTLE_NOTIFICATIONS. As seen below, it includes only one table - NOTIFICATIONS, that serves as a storage of all generated notifications sent to any user or administrator.

As stated, the NOTIFICATIONS table includes key columns which are necessary for each notification to be stored correctly. NOTIF_ID represents the identifier which can be used as a primary key of this table. The column named TYPE defines the type of the notification itself (system, message, transaction, report). On the other hand, the MESSAGE column is used to keep the actual notification text. Columns TARGET_ROLE and TARGET_USER_ID are used to indicate whether the notification should be sent to a particular user or to a certain role (admin, normal user etc.). Also, there is an additional field IS_READ which allows tracking whether the notification has been read by the recipient. Finally, the

last column called CREATED_AT shows the moment of generation of the particular notification.

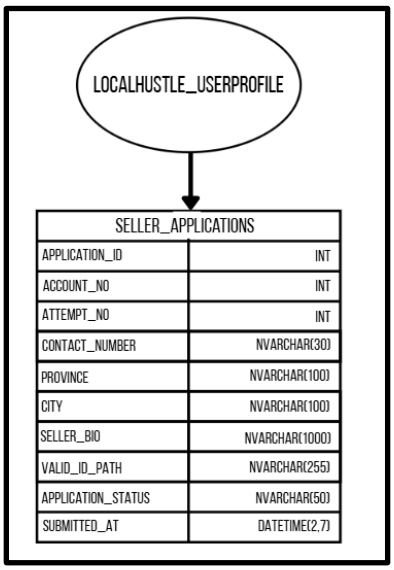


Figure 3.5.2.4 LocalHustle_UserProfile Database Structure

The figure depicts the database schema of the user profile module within the LocalHustle platform, specifically for the seller applications in the LOCALHUSTLE_USERPROFILE database. This schema comprises a single table, SELLER_APPLICATIONS, which stores all relevant data for the verification process that the application undergoes. All information relevant to the verification of applicants is contained within the SELLER_APPLICATIONS table. The APPLICATION_ID column acts as the primary key for the table, while the ACCOUNT_NO column contains the unique identifier of the account associated with the application. The ATTEMPT_NO column keeps track of the number of times an account has submitted an application. The CONTACT_NUMBER column contains contact information while the PROVINCE and CITY columns contain location

information. These columns provide necessary background information regarding the applicant.

Additional information provided by the applicants is contained within the SELLER_BIO column, while the VALID_ID_PATH column contains the URL of the ID used for identification purposes. The APPLICATION_STATUS column contains the status of the application (pending, approved, rejected), while the SUBMITTED_AT column holds the date and time when the application was submitted.

3.5.2 System Flowcharts

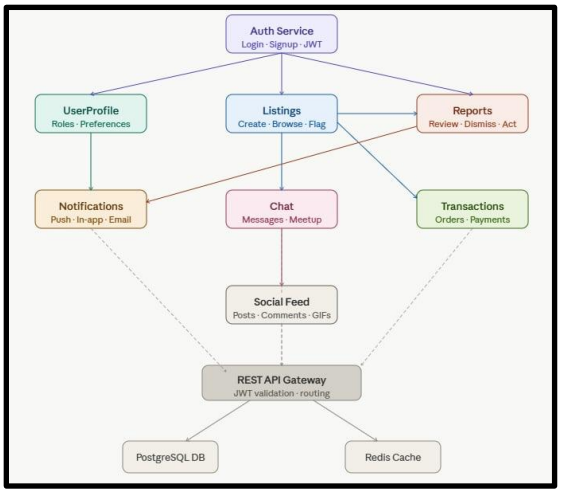


Figure 3.5.2.1 Overall Flow of Website

The following diagram describes the overall architecture of the system and its working mechanism. The starting point is the Authentication Service which deals with user authentication by logging them in and out, registering, and generating JWTs for accessing the system safely. Users who have successfully logged in can use the various modules on the application like User Profile, Listings,

Reports, Chat, Transactions, and Notifications. The Listing Module is the core feature of the app where users can create, browse, and even flag any listing.

This module is connected with the other modules like Report to handle flagged content, Chat to communicate with others, and Transactions to manage order and payment transactions. User Profiles will be responsible for managing the user profiles, roles, preferences, etc., while the Notifications module will notify users via push notification or emails. All requests made within the system go through the REST API Gateway that manages requests and also validates the JWT of the users. There will also be a PostgreSQL database along with a Redis cache for storing data and improving performance.

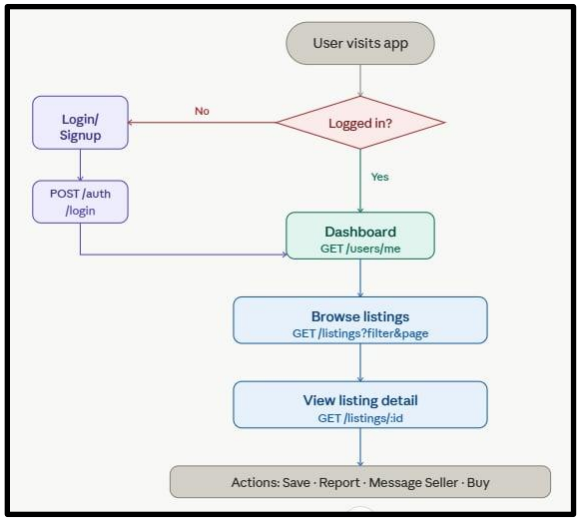


Figure 3.5.3.2 Browsing Flowchart

Above is the flowchart showing how users will use and interact with the listings within the LocalHustle site. The flow starts with a user visiting the website. From there, the process moves to determine if the user is logged into the website. If the user is not logged in, the system will redirect the user to either the login or

sign-up page, where he or she will provide his or her credentials. The system will authenticate the user and provide access, which will direct the user to the dashboard page. If the user is logged in, the system will direct the user to the dashboard page. Once the user accesses the dashboard page, he or she will click on the listings page and filter the listings. From there, the user will select a specific listing. The user will have a number of choices at hand once he or she views the listing. He or she can save the listing, report it, message the seller, or buy the listing.

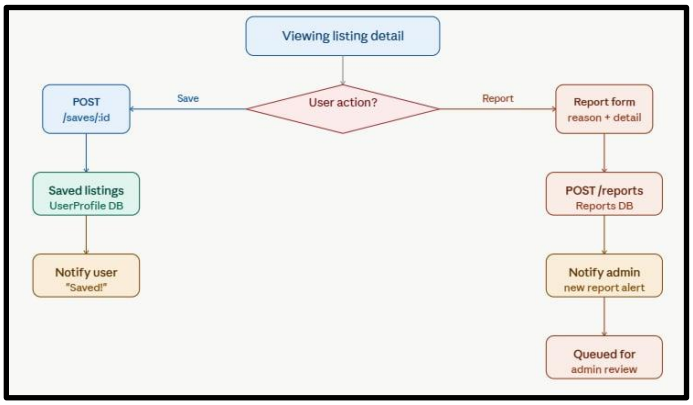


Figure 3.5.3.3 Saving and Reporting a Listing Flowchart

The flowchart above is a depiction of how a user interacts with the application following their access to a listing. Initially, a user is within the listing detail page where they make their selection of action after which the application checks whether the selection made is a save or report. In case the action taken is to save the listing, the application sends a request to save the listing in the user's saved listings database. After the request is completed successfully, the application updates the user's profile and gives a confirmation that the listing has been saved. Where the action taken by the user is to report the listing, the application guides the user to the reporting form whereby the reason and other related details are inputted. This report is then saved in the reports database. Subsequently, an alert

is sent to the administrator of the existence of a new report while the listing undergoes an administrative check-up.

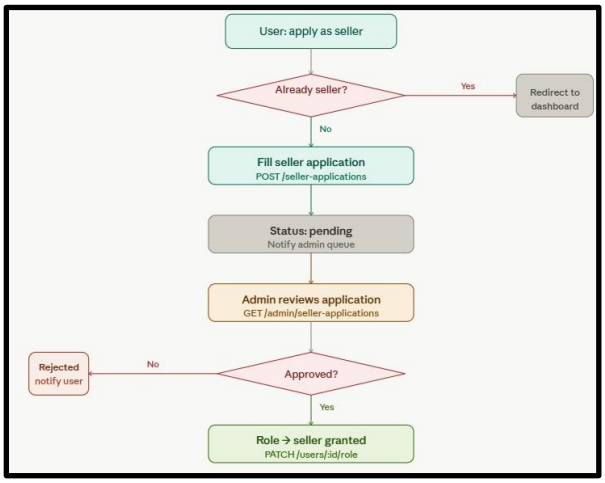


Figure 3.5.3.4 Seller Application Flowchart

Here is the flowchart depicting the process of applying for sellers on the LocalHustle platform. The process starts with an event where the user decides to apply for the role of a seller. This event takes the user to a decision-making point where their application is compared to whether they are already a seller. If they have already applied, they are led back to the dashboard. However, if they have not yet applied, they are directed to fill out the seller application form. The application form will be submitted and saved as pending in the system and entered into the review queue by the admin. The next step entails reviewing the application via the admin panel. Again, there will be a decision-making point based on whether the application has been approved or not. If approved, then the role will change to the seller, giving them seller access on the platform. However, if it has not been approved, they will be notified accordingly.

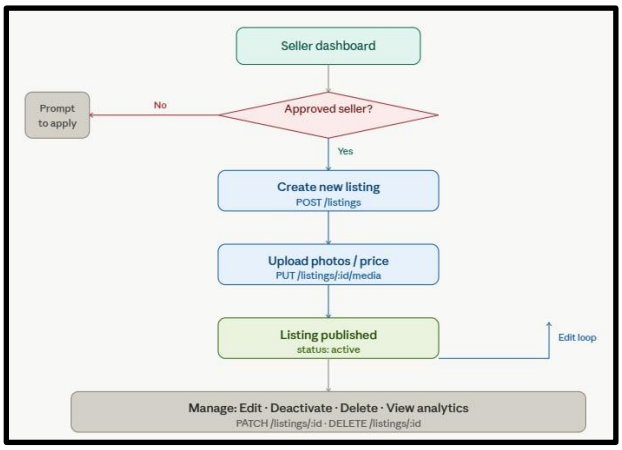


Figure 3.5.3.5 Seller Listing Flowchart

The following flowchart shows how the listing is created and managed by the sellers in the LocalHustle platform. In this process, the first thing that happens is that a user accesses the seller dashboard. Thereafter, there is a decision whether the user accessing the dashboard is already an approved seller or not. If he/she is not yet an approved seller, he/she is advised to make an application for selling before anything else.

Once the user is an approved seller, he/she is allowed to create a new listing through submission of necessary details about his/her product/service. Once the listing is created, the next step involves uploading pictures of the product/service and specifying its price. Once done, the listing will be published and thus become an active listing in the platform. Once the listing has been published, it becomes fully controlled by the seller who creates it. He/she is able to edit, deactivate, delete or even view the analysis of that particular listing. The system provides continuous edit ability of the listing.

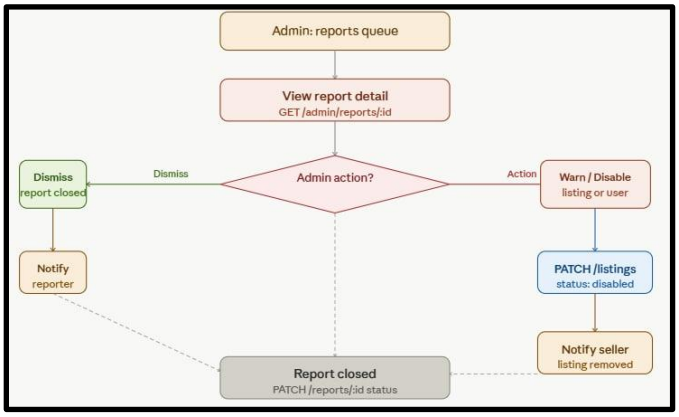


Figure 3.5.3.6 Admin Reports Flowchart

The following diagram shows the process for handling listings reported by users in the LocalHustle website. To begin with, the admin navigates to the reporting queue to view all pending reports made by users. Next, the admin chooses a particular report and checks the details of the report such as the reason for the reporting and any relevant details that have been shared by the user. The next step involves making a decision by the admin depending on whether the report is invalid or valid. If the report is found invalid, the system will mark the report as closed and notify the user accordingly about the decision made regarding their report. Conversely, if the report is valid, the admin can take necessary action against either the listing or the user. The status of the listing will be updated and a message sent to the seller regarding the decision made.

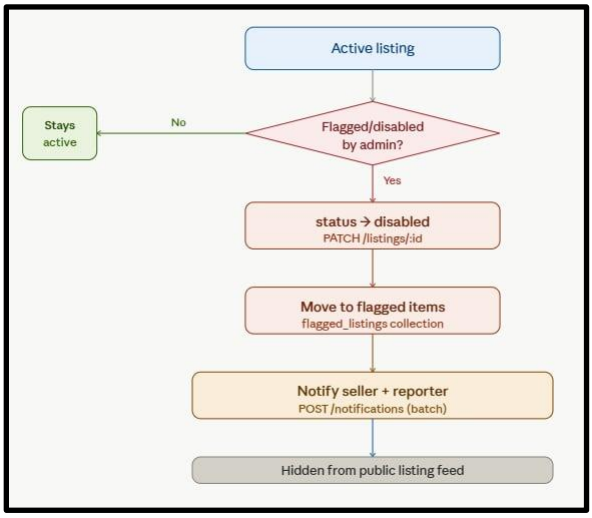


Figure 3.5.3.7 Flagged Items Flowchart

This flowchart illustrates how the LocalHustle platform manages listings that may be flagged or disabled by the administrator. The process begins with an active listing that is currently visible to users in the public feed. A decision point then checks whether the listing has been flagged or marked for action by the admin. If no issues are found, the listing remains active and continues to be accessible to users without any changes.

However, if the listing is flagged or determined to violate platform guidelines, the system proceeds to update its status to disabled. Once disabled, the listing is moved to a separate flagged items collection for tracking and administrative reference. This ensures that flagged content is properly recorded and can be reviewed if needed. The system then sends notifications to both the seller and the reporting user, informing them of the action taken and the current status of the listing. Finally, the listing is removed from the public listing feed, preventing further visibility or interaction. This process ensures proper moderation, accountability, and maintenance of content quality across the platform.

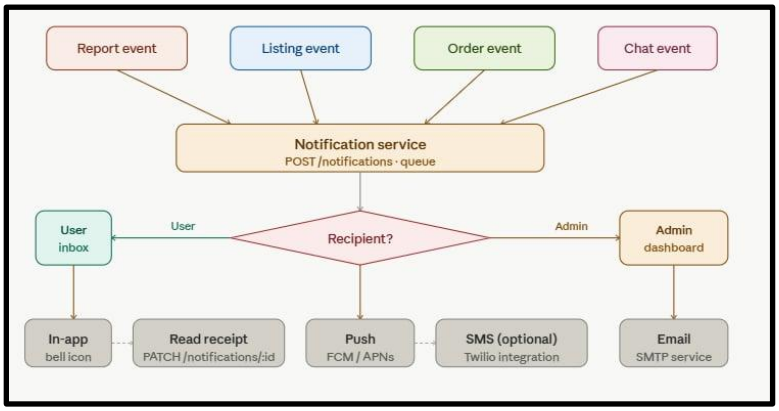


Figure 3.5.3.8. Notification Service Flowchart

This diagram demonstrates the flow of events regarding how the notification system works within the LocalHustle platform. It all starts when various events happen in the system; they may include reports being filed, listings taking place, orders being updated, and even chatting. This prompts the notification service to handle the event by sending the notifications queue.

Once the event happens, the destination is determined based on the recipient of the notification; it could be either a user or an admin. Should the recipient be a user, then the notification is sent to the user's inbox where it is sent out as an app notification that could take the form of an icon like a bell. Moreover, the reading status of the notification could be checked from the system depending on the settings of each notification. The notifications could also be sent out as push notifications using FCM and APNs with the option of integrating the system using SMS integration. In case the notification is meant for an admin, it is sent to the admin dashboard and also to their email addresses via SMTP service.

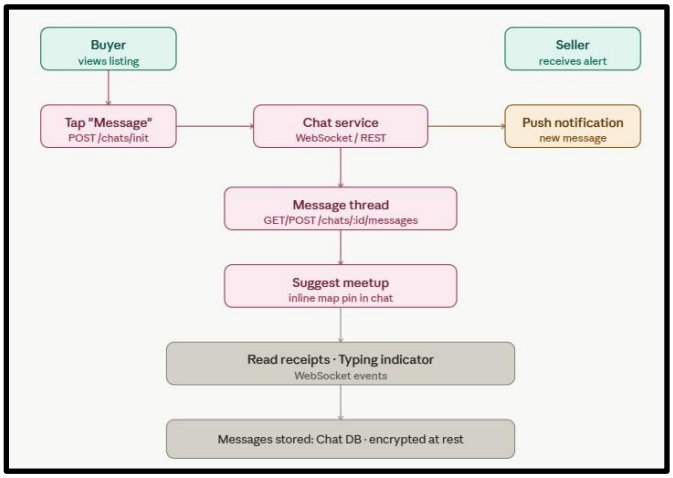


Figure 3.5.3.9. Chat Service Flowchart

The diagram shows how the chat system works in LocalHustle, allowing buyers and sellers to communicate. It starts when a buyer looks at the product being sold and clicks on the "Message" button to start the discussion. This event leads to the initiation of the chat system that uses WebSocket and REST protocols for instant or almost-instant messaging. Once the chat is started, the message thread will be set up, and both parties involved can send and receive messages. Meanwhile, the seller gets notified about the new message, which leads to his reaction to the received message. Inside the chat window, users can talk with each other and even discuss potential meetups by using tools such as the map pin. The chat system can notify the user that another person is typing, providing additional information in real-time. All messages sent in the chat are saved inside the chat database in encrypted form.

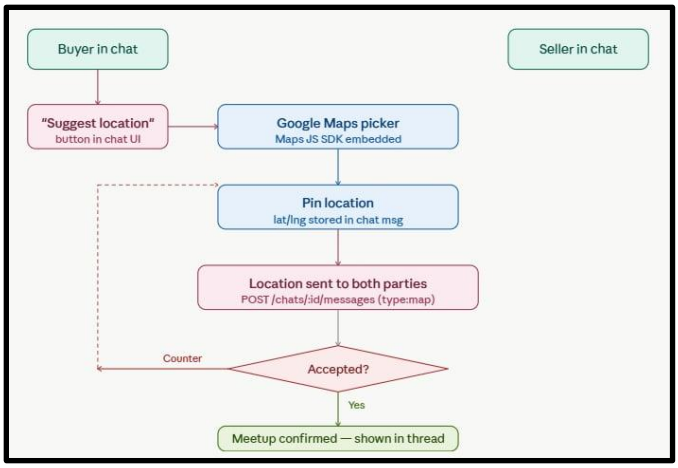


Figure 3.5.3.10. Meetup Service Flowchart

The diagram provides the flow chart of how the meetup location feature functions in the chat system of the LocalHustle platform. The process starts when a buyer opts to use the feature by selecting "Suggest location" option in the chat interface. As a result, the user will be presented with a Google Maps picker to select the particular location. After selecting the location, it will be pinned and recorded as part of the chat message together with the latitude and longitude. Then, the pinned location will be shared with both the buyer and seller in the same chat message, which means that it will become available for the two users. At this point, a decision point will emerge to either accept or suggest another location. In case the location is not accepted, the cycle of selecting the new location continues; otherwise, the meetup will take place and its confirmation will be sent via the chat.

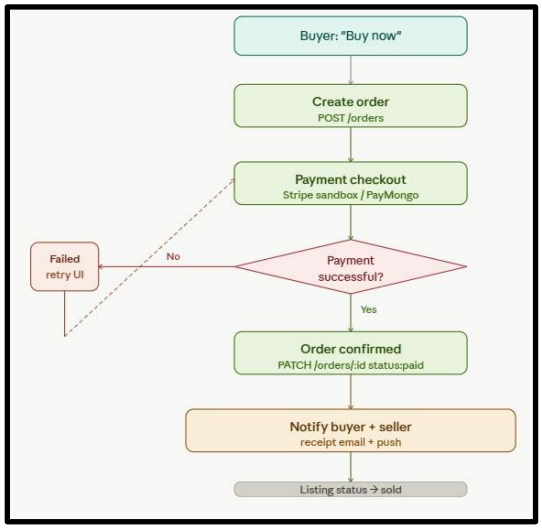


Figure 3.5.3.11. Payment Flowchart

The above flowchart shows the purchasing and payment flow within the LocalHustle app. The purchasing flow starts when the buyer clicks the "Buy Now" button on the listing. It will create an order for the product, after which the payment flow will start, where the user will make payments using the payment systems that are connected with the app. There will be a decision point where the system will determine if the payment was successful.

In case there is no successful payment, the app will redirect the user to the "retry" page and allow them to try again. It is crucial since it will give buyers multiple chances to pay without having to repeat the entire process. In case there is successful payment, the system will update the order status to "paid." It indicates that the transaction was successful. Once the transaction is completed, both the seller and buyer will receive emails and notifications about the transaction and receipts. Lastly, the listing will be marked "Sold" on the app, and it will not be possible for other users to buy the product anymore.

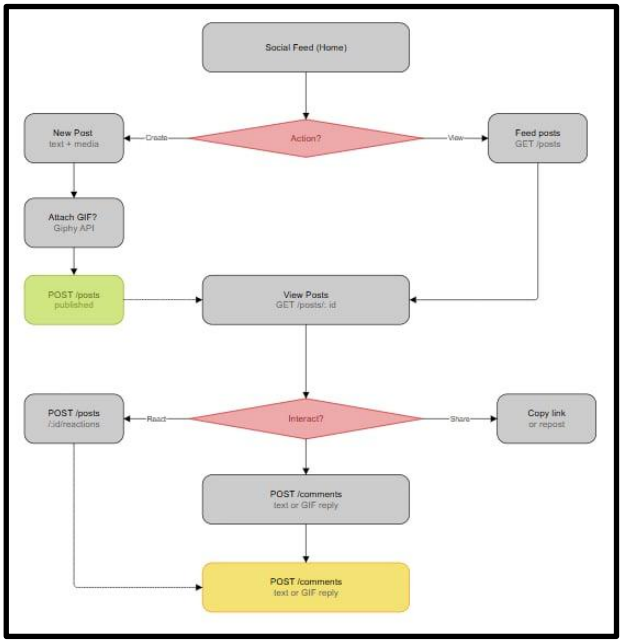


Figure 3.5.3.12. Social Feed Flowchart

The above flowchart highlights how the social feed functionality works in the LocalHustle application, enabling users to engage in creating and viewing different posts. The process starts from the social feed homepage, where a decision needs to be made by the user on whether he would like to create new content or view any available posts on the feed. Should the user decide to create his content, he gets the option to write new content on the feed by composing some text and adding media such as GIFs using Giphy API.

In case the user opts to view the available posts, then the feed is presented for him to view. The selected post by the user is then viewed to enable him to choose the action that he would want to undertake in relation to the post.

Interacting with the selected post gives the user the ability to react to the post, add comments using text or GIFs, or even share by copying the content's link or reposting.

3.5.3 UML

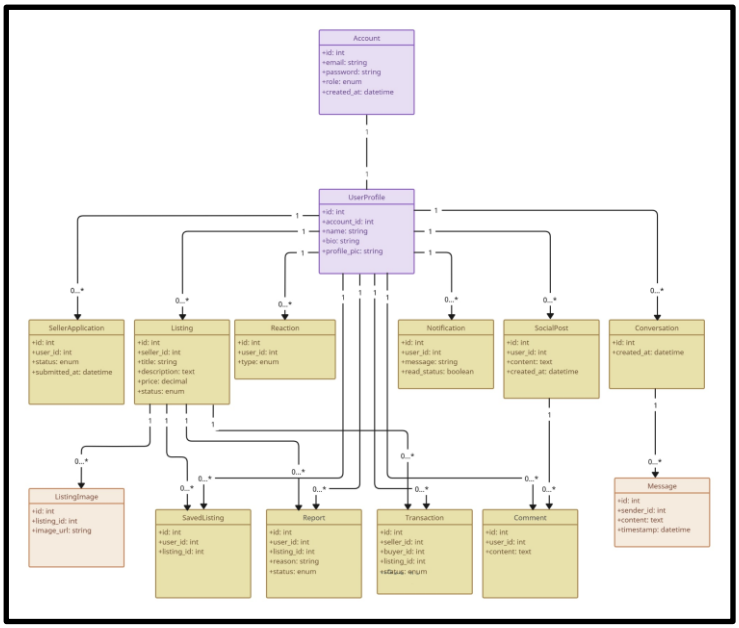


Figure 3.5.3.1 Class Diagram

This diagram shows the logical database design of LocalHustle. First and foremost, the central entity for all the actions in LocalHustle is the Account. This entity holds crucial user data including but not limited to their email address, password, role in the app, and the account's creation date. The Account entity has a one-to-one relationship with a UserProfile where other user data like their name, description of themselves, and profile image can be stored.

This central entity then has multiple relations with other entities through one-to-many relationships to accommodate all the actions a user might take in the app. These include creating multiple SellerApplications, multiple Listings, reacting to Listings, receiving Notifications, creating SocialPosts, and having multiple conversations on the app. Furthermore, each listing created by the sellers can contain multiple ListingImages and these listings can also be bookmarked by the user through the SavedListing entity. The design of the system includes moderation mechanisms and transaction processes. Users have an opportunity to report inappropriate or fraudulent listings using the Report object that allows managing content and performing administrative tasks.

Transactions made by users are captured using the Transaction object, connecting buyers and sellers with their corresponding listings. Social activity among users is supported by SocialPost and Comment objects that allow posting and commenting on content created by users. Communication processes among users are managed using Conversation and Message objects where conversations include numerous messages from users. In general, the above-mentioned diagram demonstrates how well-organized and connected a system is since the UserProfile object serves as the core element of a platform that manages users' actions in marketplace, social, and communication functionalities.

3.6 System Components

3.6.1 Software Components

The LocalHustle web application is created using the combination of known core technologies working together to provide the intended functionality as well as user experience. For the backend logic, it was built using PHP, handling server-side logic like user authentication, data processing, and interaction with other

databases. When it comes to building the front-end, HTML is utilized to organize content layout and then is refined and styled using CSS to help capture and maintain the main vibe and consistent interface color schemes and look of the system across each page. Lastly, JavaScript is applied to improve interactivity and responsiveness of the system, which then allows it to gain dynamic features such as real-time form validation, interactions with modals, and overall smoother navigation. These four technologies form an efficient software stack that basically made the implementation of the system's marketplace and social features possible.

3.7 Ethical Considerations

The study conducted will be in line with existing ethical principles that ensure the safety, privacy, and rights of all individuals who participate in the assessment of the LocalHustle system. The subjects will be adequately briefed about the aims and objectives of the investigation and the way in which the data obtained will be utilized. Participation in the investigation will be completely voluntary and subjects may choose not to take part or withdraw from the investigation at any point in time.

Furthermore, in order to protect the privacy of subjects, they will be asked to provide no personally identifiable data in their responses to the survey questions. In addition, all data obtained from the study will be used for academic purposes only and the privacy of participants will be fully guaranteed. Moreover, the research will make sure that no harm, pain, or risks are caused to the subjects in the course of collecting data. The tests will involve the use of the LocalHustle website under normal conditions only. Besides, the system features will be tested in a controlled environment so that no harm is done to the database.

REFERENCES

Abgaz, Y., McCarren, A., Elger, P., Solan, D., Lapuz, N., Bivol, M., ... & Clarke, P. (2023). Decomposition of monolith applications into microservices architectures: A systematic review. *IEEE Transactions on Software Engineering*, *49*(8), 4213-4242.

Bokoliya, M. (2023). *Impact of digital platforms on local economies* [Unpublished manuscript]. Department of Geography, Indraprastha College for Women, University of Delhi.

ECommerce sales & size Forecast. (n.d.). International Trade Administration | Trade.gov. https://www.trade.gov/ecommerce-sales-size-forecast

Haganus, C. P., Jr., Huring, C. B. B., Caballero, D. K. B., Cano, K. C. E., Abing, N. G., & Ilusorio, G. L. (2017). Responses of vendors on trust issues arising from online selling. *Journal of Multidisciplinary Studies, 6*(2), 47–60. https://doi.org/10.7828/jmds.v6i2.1044

Laudon, K. C., & Traver, C. G. (2020). *E-Commerce 2020–2021: Business, Technology and Society, eBook, Global Edition*. https://elibrary.pearson.de/book/99.150005/9781292343211

Shabani, I. (2022). Design of modern distributed systems based on microservices architecture. *International Journal of Advanced Computer Science and Applications*.

Terdal, S. (2022). Microservices enabled e-commerce web application. *International Journal for Research in Applied Science and Engineering Technology*, *10*(7), 3548-3555.